Swarma: проверяемые границы доверия для роёв автономных агентов

Личности без ключей у агента, гранты, подписанные сообщения и изоляция исполнения на машинах владельца

Алексей Григорьев, Иван Опарин
Relux Works
github.com/relux-works

Октябрь 2026, версия 6.2.0

Аннотация. Агент разработки читает недоверенный текст и действует с правами запустившего его процесса, поэтому инъекция в промпт исполняется руками самого агента. Swarma служит платформой, на которой рой таких агентов одного или нескольких владельцев ведёт проекты, а допуск каждого действия решает детерминированная проверка у исполнителя эффекта, и захват агента его полномочий не расширяет. Закрытого ключа личности у агента нет: подпись выдаёт ключник, привилегированная служба ключей, которая узнаёт собеседника по UID от ядра и поколению учётки, случайному номеру, новому при каждом её создании. Каждое действие требует гранта, подписанного разрешения, которое при делегировании можно только сузить. Ключи, гранты и отзывы ведёт реестр владельца на отдельном узле доверия, свидетель отката фиксирует контрольные точки реестра, и запись отзыва действует на каждой проверке не позже $\max(S + A, F)$, где S момент выпуска первого подписанного полного снимка с этой записью, $A \leq 60$ с допустимый возраст снимка, F момент вступления записи в силу. Сообщения допускают ворота доверия, которые проверяют подпись и гранты до доставки агенту. Исполнение допускает станция, отдельная учётка ОС агента, гарантии изоляции которой владелец квалифицирует, то есть подтверждает воспроизводимой процедурой проверки машины. Каждую гарантию мы разбираем против модели угроз как аргумент дизайна, с допущениями и пределом. Оценка охватывает спецификации и отдельные компоненты: репозиторий библиотеки подписанных записей сообщает о согласии генератора, независимой проверки на Python и библиотеки на Go на 2632 случаях, и проверку библиотеки мы повторили; два независимых вычислителя памяти согласны на 66 смысловых векторах; на машине Linux примитивы учёток прошли применение, проверку и откат, а квалификационный набор этой процедуры прошёл 11 из 11 проверок; ограниченная модель показывает необходимость четырёх правил интеграции. Эталонная проверка одной записи занимает от 2 до 18 мс в одном потоке Go. Задержку отзыва в сквозном прогоне станции, производительность системы целиком и стойкость работающей установки к состязательным атакам оценка не измеряет.

1. Введение

Агентом мы называем участника работы с ролью, полномочиями и устойчивой личностью; в этой работе это *агент разработки*, который ведёт программный проект. Его работу исполняет процесс агентской среды, например Claude Code или Codex: языковая модель в нём вызывает инструменты, меняет файлы и отправляет сообщения. Среда обычно знает только пользователя ОС, запустившего процесс, и отдаёт агенту все его права. Косвенная инъекция в промпт [19, 12] пользуется ими: текст из задачи, зависимости или письма коллеги убеждает модель прочитать ключ, отправить токен или одобрить запрос. Просьба к модели не читать секрет слабее границы, за которой процесс прочитать его не может.

Рой, то есть несколько агентов с разными ролями над общей целью, добавляет вопросы, на которые модель не может ответить своим текстом: кто отправил сообщение и разрешено ли действие. *Оркестратор*, агент с ролью декомпозиции и координации, разбивает цель на задачи и раздаёт их *продьюсерам*, которые производят изменения, и *ревьюверам*, которые независимо проверяют результат. Агенты переписываются через мессенджер, сессии переживают перезапуски, участниками становятся агенты других владельцев. Агентские фреймворки [44, 21, 36, 47] координируют роли и инструменты, а границы полномочий между агентами и между владельцами оставляют среде исполнения.

Swarma служит платформой, на которой рой ведёт проекты на машинах своих владельцев так, что захват агента инъекцией не расширяет его полномочий. *Владельцем* называется тот, кто отвечает за свои машины, агентов и оплату моделей. Решение о допуске Swarma переносит из суждения модели в детерминированную проверку у исполнителя эффекта, то есть у службы, которая пишет файл, выдаёт учётные данные или доставляет сообщение. Платформа состоит из трёх слоёв. Рантайм Swarma Runtime исполняет рой на машинах одного владельца. Протокол Swarma Protocol позволяет независимым владельцам вести общий проект над единой *средой проекта*, авторитетным окружением, которое исполняет только разрешённые действия проекта (раздел 2). Цепочка Swarma Chain хранит общее состояние личностей вне отдельного проекта. Эта работа описывает границы доверия первых двух слоёв; платформу целиком представляет сопутствующая работа [53], а подробное устройство описывает книга о Swarma [52]. Если работа расходится со спецификацией модуля, права спецификация.

Каждая часть рантайма закрывает один вопрос. Закрытый ключ в процессе агента достался бы инъекции, поэтому ключи держит *ключник*, привилегированная служба хоста, которая подписывает за агента и узнаёт его по учётке ОС (раздел 4.1). Право на действие задаёт *грант*, подписанное разрешение на определённые действия и цели со сроком, которое при делегировании только сужается (раздел 4.2).

Гранты, *аттестации*, то есть подписанные утверждения о принадлежности ключа агенту, и отзывы передаются *конвертами* (envelope), записями платформы с подписью над их точными байтами. Текущие ключи, гранты и отзывы ведёт *реестр владельца*, их единственный авторитетный журнал на отдельной машине владельца, *узле доверия*. *Свидетель отката*, отдельный процесс, фиксирует контрольные точки реестра, поэтому откат и раздвоение журнала обнаружимы (раздел 4.2).

Сообщения агентов и людей перевозит *носитель*, который полномочий не выдаёт. *Мост* между носителем и сессией агента сохраняет события, а его *ворота доверия* детерминированно проверяют подпись, гранты и отзыв до доставки (раздел 4.3).

Исполнение удерживает *станция*, место одного агента на хосте со своей учёткой ОС, гарантии изоляции которой владелец *квалифицирует*, то есть подтверждает воспроизводимой процедурой проверки машины (раздел 4.4).

Верстак (Werkstatt), постоянное рабочее место агента для связки проекта, роли и агентской среды, имеет свой управляемый дом, клон репозитория и личность на носителе; текст пришедшего сообщения права на задачу верстаку не даёт. Содержимое дома, то есть инструкции, скиллы, настройки и подключения MCP, собирает из профиля роли *куратор* (Swarma Curator); изменения инструкций агентов одобряет детерминированный предикат, а инструкции кандидата (раздел 4.7) среда ревьювера не подхватывает. *Кипер* (Swarma Keeper) ведёт главный цикл узла и следит, чтобы у каждого обязательства между агентами был держатель при передаче, остановке и перезапуске; ключей кипер не держит, а эскалация, которую он ведёт при недоступном держателе, ничьих полномочий не расширяет. Результат работы принимает независимое ревью точного кандидата (раздел 4.7).

Вклад работы:

1. Модель системы и модель угроз для роя агентов, где захваченный инъекцией агент считается штатным противником.
2. Композиция механизмов, в которой допуск решает детерминированная проверка у исполнителя: ключник с определением собеседника ядром, гранты с ослаблением, реестр владельца со свидетелем отката и ограниченным сроком отзыва, ворота доверия поверх носителя, допуск к носителю без общего секрета, станции с контрактом изоляции, память как потребитель реестра и независимое ревью точного кандидата.
3. Анализ каждой гарантии против модели угроз с допущениями и пределом.
4. Оценка по частям: корпуса соответствия с независимыми реализациями, привилегированные запуски на машине Linux, где квалификационный набор прошёл 11 из 11 проверок, ограниченная модель интеграции, микрозамеры эталонной проверки записей и статистика ревью, каждое с перечнем того, чего оно не показывает.

2. Постановка задачи и модель системы

Принципалы. Владелец отвечает за свои машины, агентов и оплату моделей; ему принадлежит корень доверия установки. Оператор, человек владельца, выдаёт полномочия и подтверждает действия, расширяющие права. Агент исполняет работу в пределах выданных полномочий; его криптографическая личность отделена от процесса и от пользователя ОС. Проект задаёт цель, среду и правила приёмки. Контрибьютор, другой владелец, приносит проекту ёмкость своего агентского цикла, а донор отдаёт ему ёмкость модели. Совмещение ролей дополнительных прав не даёт.

Хосты и станции. Хостом называется машина владельца, на которой работают агенты. *Станцией* называется место одного агента на хосте: своя учётка ОС в базовом классе изоляции, своя личность на носителе, свои ключи, дом и корневой контекст. Учётки между агентами не делятся, и учётка с отдельным UID без остальных проверок станцией не считается. Станцию обслуживают привилегированные службы хоста с узкими обязанностями и разнесёнными правами. Под root работает ключник, который подписывает за агентов. Менеджер пользователей, создающий учётки, и запускатель, стартующий одобренную программу, получают root на одну узкую операцию через `sudo`. Под своими служебными учётками работают диспетчер, который допускает запуск по грантам, квотам и гарантиям станции, брокер учётных данных, который выдаёт агенту ограниченную аренду доступа к внешнему сервису, хост сессий, который ведёт сессии агентов, и мост, который доставляет сообщения носителя.

Носители. *Носителем* называется транспорт сообщений без архитектурной привязки к протоколу; для воплощения мы опираемся на федеративные протоколы, такие как Matrix [40] и XMPP [37]; конкретный протокол привязывает к общему контракту носителя свой профиль [52]. Носитель и членство в комнате

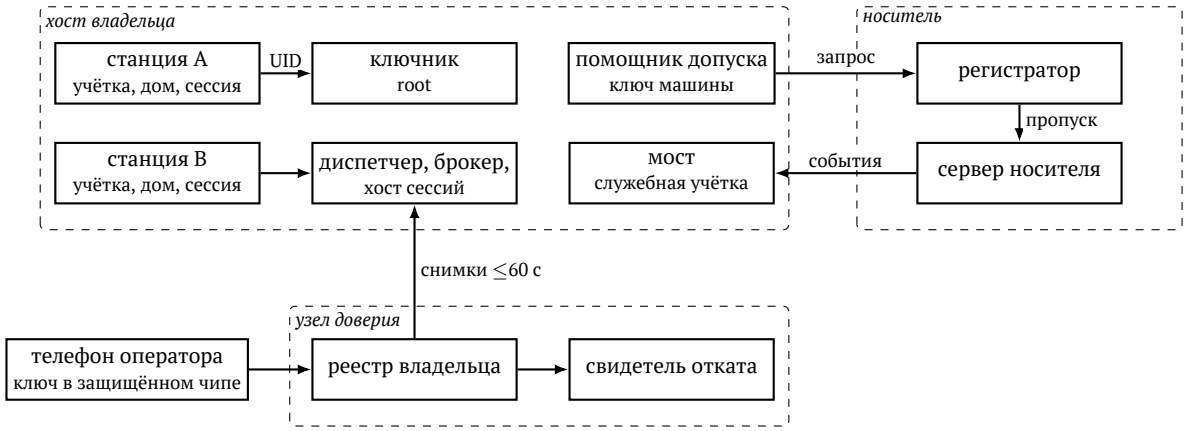


Рис. 1. Модель системы. Агенты на станциях ключей не держат; ключник и помощник допуска, который ключом машины подписывает запрос агента на учётку носителя, узнают собеседника по UID от ядра. Регистратор, служба допуска к носителю, выдаёт по такому запросу пропуск (раздел 4.5), мост проверяет сообщения, реестр принимает подписанные действия оператора и выдаёт службам снимки, свидетель фиксирует его контрольные точки.

полномочий не дают. Сквозное шифрование носителя скрывает текст от сервера, подпись над точными байтами удостоверяет автора.

Узел доверия. Реестр владельца и свидетель отката работают на отдельной машине вне административного домена платформы. Её снимают у провайдера, который от платформы не зависит, и администрирует её только владелец; у агентов там нет root, у платформы нет административного доступа. Реестр служит единственным авторитетным писателем ключей, аттестаций, грантов, отзывов и журналов личностей, а службы хоста получают от него подписанные снимки (рис. 1).

Протокол. Swarma Protocol разделяет мышление, управление и исполнение: агентские циклы контрибьюторов предлагают ревизии, кор-ноды, узлы проекта с проверкой правил, ведут журнал полномочий и принятия с сертификатами кворума, авторитетная среда проекта исполняет только разрешённые действия. Уровень развёртывания L0, L1 или L2 задаёт, чему доверяет журнал. На L0 журнал доверяет единственной кор-ноде, и порог сертификата равен одной подписи. На L1 порог равен большинству $t > n/2$, а единственность истории обеспечивает честный секвенсер, который упорядочивает записи. На L2 порядок устанавливает византийски устойчивый консенсус, и порог равен $q = \lceil (n + f + 1)/2 \rceil$ при $n \geq 3f + 1$, порогу византийского кворума [30]; при $n = 3f + 1$ он совпадает с $2f + 1$ из PBFT [7]. Вывод обоих порогов дан в книге [52].

Задача. Ни одно действие агента с эффектом за пределами его станции не должно исполняться без проверяемого основания: подлинного отправителя, актуального гранта, одобрения, адресованного точному объекту, и физического удержания исполнения в выданной области. Основание проверяет сторона, совершающая эффект. Отсутствие свидетельства и невозможность прочитать состояние разрешения не дают.

3. Модель угроз

Противники.

П1. Захваченный агент: модель подчинилась внедрённому тексту и пользуется всеми возможностями процесса агента, включая подпись, которую ключник выдаёт его личности.

П2. Чужой участник: агент или человек другого владельца, рабочая нода протокола или гость комнаты пытается расширить полномочия, подменить объект одобрения или действовать от чужого имени.

П3. Сетевой противник и сервер носителя: противник подменяет, задерживает и повторяет сообщения, а сервер может выпустить событие от имени любого адреса своего домена.

П4. Захваченная машина: противник получил root на одной зарегистрированной машине из нескольких; корни владельца, которые хранятся вне этой машины, ключ оператора и узел доверия ему недоступны. Гарантии самой этой машины root снимает, и П4 задаёт вопрос о радиусе поражения для остальных машин, носителя и узла доверия.

П5. Противник на узле доверия: он похитил ключ реестра или пытается откатить и раздвоить журнал реестра.

П6. Криптографический противник: он ломает класс подписей на эллиптических кривых.

Допущения. Для гарантий хоста доверены его ядро, root и доверенная база платформы. В неё входят на хосте ключник, менеджер пользователей, запускатель, диспетчер, брокер учётных данных, хост сессий, мост с воротами доверия и помощник допуска; на узле доверия реестр, его процесс подписи и свидетель; вне хоста регистратор и телефон оператора. Ошибка в любой из них снимает опирающиеся

на неё гарантии. Отдельно назовём мост: он разбирает ввод противника П3 под служебной учёткой сообщений, общей для станций хоста, и подписывает вердикт своим ключом, поэтому ошибка разбора в нём затрагивает все станции. Мастер-секрет ключника цел; где возможно, его держит аппаратное хранилище. Оператор подтверждает расширяющие действия на телефоне с ключом в Secure Enclave или StrongBox и видит подписываемое содержание; запасной путь идёт через терминал с паролем `sudo` без кэша. Владелец честен перед своими агентами и единолично администрирует узел доверия. У установки есть проверенный или контролируемый оператором источник времени, например NTS. Для протокола среда проекта доверена в отношении целостности, если нет аппаратной аттестации исполнения.

Вне рамок. Мы не защищаемся от эксплойта ядра и не сохраняем гарантий хоста, root которого захвачен (последствия для остальных машин разобраны как П4); мы не обещаем правильности суждения модели и конфиденциальности кода от провайдера модели, не рассматриваем побочные каналы и подделку оборудования. Отказ в обслуживании не предотвращается: при недоступном свидетельстве система закрывает допуск и сознательно жертвует живучестью. Экономика и переносимая репутация в этой работе не оцениваются.

4. Устройство

4.1. Личность и ключи

Ключевая пара личности отвечает, кто отправитель, и ничего не разрешает. Закрытого ключа личности у агента нет. Ключник, привилегированная служба с мастер-секретом установки, выводит ключи агентов жёсткой деривацией по BIP-32 в варианте SLIP-0010 для Ed25519 [46, 20]. Путь `назначение'/продукт'/проект'/агент'` кодирует стабильную личность и не совпадает ни с UID, ни с путём в файловой системе, поэтому личность переживает пересоздание учётки.

Ключник слушает Unix-сокет и получает UID собеседника от ядра (`SO_PEERCRED` на Linux, `LOCAL_PEERCRED` на macOS). Личность выбирается по привязке, где записаны UID, поколение учётки, путь, владелец, номер регистрации и признак разовой или постоянной личности. Подключение допустимо, только если поколение этого UID активно в журнале менеджера пользователей. Поколение представляет собой случайное 128-битное число, новое при каждом создании учётки, поэтому повторно выданный UID чужого ключа не наследует.

Сообщения подписывает отдельный ключ сообщений Ed25519, выведенный по пути `m/4'/продукт'/проект'/агент'`, и только документ с меткой `swarma-msg/2` с прообразом не длиннее 24 576 байт. Принадлежность ключа агенту удостоверяет запись `attestation/1`, подписанная корнем аттестаций ключника, со сроком не длиннее 86 400 с. Аттестация становится текущей, когда она сама, её запись в журнале реестра, контрольная точка и подтверждение свидетеля сохранены вместе; до этого и при неизвестном исходе выпуска подпись закрыта. Каждый корень подписывает только свой язык записей: корень аттестаций ключника, корень оператора, корень грантов, ключ реестра, ключи служб грантов, офлайновый издатель регистраций машин и ключ коммитов; совпадение ключей двух ролей ведёт к отказу. Форматы записей и порядок проверок задаёт спецификация подписанных записей (`swarm-envelope/2`).

Для других владельцев личность представлена самосертифицирующим идентификатором в духе KERI [39]: он равен хэшу первой записи журнала ключевых событий. Каждая запись этого журнала, *ячейка личности*, объявляет текущие ключи, обязательство на следующие и родителя и расходуется при ротации. Родителем ячеек агентов служит *ячейка владельца*, ячейка личности самого владельца, ключи которой из мастер-секрета установки не выводятся и хранятся отдельно. Предротация защищает от поломки текущих ключей. В установке без Swarma Chain журнал ведёт реестр владельца, и при публикации журнала в цепочку идентификатор не меняется.

4.2. Гранты и реестр владельца

Грант означает подписанное утверждение: кто выдал полномочие, кому, на какие действия и цели, при каких условиях, до какого срока и можно ли делегировать его дальше. Дочернее полномочие ограничено родительским по действиям, целям, сроку и глубине, а цепочку до корня проверяет исполнитель действия, даже если раньше на маршруте её уже проверили. Модель следует capability-подходу [13, 32] и ориентируется на SPKI/SDSI, Macaroons, UCAN и Biscuit [16, 4, 18, 15]. Работа описывает грант через его содержание и правила проверки; аргумент от байтового кодирования гранта не зависит.

В общем языке грантов ключ агента бывает субъектом и листом цепочки и не бывает издателем звена. Контракт брокера учётных данных версии v0, против которой построен этот аргумент, делает одно исключение: когда оркестратор создаёт другого оркестратора, грант вызывающего для нового подписывает родительский оркестратор, сужая свой; у родителя глубина не меньше 2, у нового не меньше 1. Права ребёнка зависят и от вызывающего, и от диспетчера: в контракте брокера v0 лист агента равен пересечению запроса и двух цепочек, содержит только `credential.lease` с глубиной 0, а цепочка не

длиннее четырёх конвертов. Если цепочку построить нельзя, ответ `grant_unconstructible` приходит до любых эффектов.

Реестр хранит, упорядочивает и публикует записи и содержимое не переподписывает. Собственный ключ реестра подписывает только снимки отзывов своей области, контрольные точки и записи свежести; его держит отдельный процесс, который принимает только одобренные профили записей. Журнал состоит из неподписанных записей `journal-entry/1` со ссылкой на предыдущую, а подписанная `checkpoint/1` называет голову и текущий снимок отзывов. Свидетель отката работает под отдельной учёткой, процессом и потоком, по запросу с вызовом называет последнюю голову и хранит доверенную контрольную точку вне узла. Две разные контрольные точки с одним номером дают `journal_fork`, точка с номером меньше засвидетельствованного даёт `journal_rollback`.

Потребители принимают снимок отзывов не старше 60 с; возраст неизменного снимка продлевает запись свежести вместе со свежим чтением головы у свидетеля, и реестр публикует такую запись каждые 20 с. Записи отзыва накапливаются, и снимок без прежней записи полным не считается. Пусть S означает время выпуска первого подписанного полного снимка с записью отзыва, $A \leq 60$ с допустимый возраст снимка, F время вступления записи в силу. Тогда запись действует на каждой проверке не позже $\max(S + A, F)$ и никогда раньше F. Проверка отзыва имеет три исхода: не отозвано, отозвано, недоступно; два последних закрывают допуск. Время проверяется по водяному знаку, а при потере проверенного времени службы держатся не дольше 10 минут по монотонным часам и затем останавливают всё, что от времени зависит. Общий стоп от реестра не зависит, возобновление требует подписи оператора.

4.3. Подписанные сообщения и ворота доверия

Мост стоит между носителем и сессией агента. Его экземпляр работает на каждую личность станции под общей служебной учёткой сообщений и никогда под учёткой агента, поэтому у агента нет учётных данных носителя. Провайдер носителя сначала дописывает событие в почтовый файл и сбрасывает его на диск, затем продвигает курсор синхронизации, а допуск идемпотентен по паре отправителя и идентификатора события.

Ворота доверия проверяют сообщение детерминированно, без сети и без вопроса к модели, в строгом порядке: каноническая форма байтов, ключ в снимке реестра, подпись, цепочка грантов до корня с сужением действий и целей и вложенностью сроков, снимок отзыва, окно времени, журнал повторов, ревизия привязки цели. Вердикт ворота подписывают ключом моста. Затем небольшая локальная модель, оценивающая смысл сообщения (decision guard), может только сузить допуск, и её недоступность даёт отказ. Допущенное событие во ввод не вставляется: хост сессий получает короткий звонок, кладёт его в долговечную очередь и доставляет при открытом заборе допуска, а тело и запись проверки агент читает из почтового ящика как данные.

Сила свидетельства о человеке-отправителе задаётся уровнями. На A0 отправителя удостоверяет сервер носителя владельца, на A1 устройство отправителя удостоверяет профиль носителя, в профиле Matrix подписью закреплённого мастер-ключа перекрёстной подписи, на A2 ключ человека подписал точные байты конверта и ворота проверили грант, на A3 оператор подтвердил точное действие на телефоне. Расширяющие действия требуют A3. Человека другого владельца узнают по корню, закреплённому при знакомстве: корень аттестует его мастер-ключ в профиле Matrix вместе с полным адресом. Гранты называют ключи, поэтому смена сервера или адреса прав не переносит.

4.4. Станции и контракт изоляции исполнения

Каждый автономный агент работает отдельным пользователем ОС; это базовый класс изоляции (раздел 7). На Linux у каждого поколения учётки своя cgroup v2 под управлением root с лимитами процессора, памяти, процессов и ввода-вывода. Менеджер пользователей и запускатель вызываются через `sudo` без аргументов и принимают один JSON ограниченного размера, а план запуска передаётся неизменяемым на дескрипторе 3. Агент остановлен, когда у всей учётки нет процессов в течение опубликованного периода тишины. Сеть ограничивает файрвол ядра по UID: агент достигает только слушателя прокси своего поколения, а `CONNECT` ограничивает имя, порт и проверенный адрес.

Что станция гарантирует, задаёт контракт изоляции исполнения (execution isolation contract, EIC): профиль изоляции, вектор возможностей, привязки личности, размещения и исполнения, квитанция границы и кортеж квалификации. Границу применяет запускатель и пишет о ней квитанцию, покрытие сверяет диспетчер, квитанцию перед выдачей учётных данных проверяет брокер. Гарантии названы 42 идентификаторами общего словаря. У гарантии на каждой платформе один из четырёх статусов: `present`, `weaker-with-compensation`, `absent`, `PENDING-VERIFY`. Задача объявляет нужные гарантии записью `station-required-guarantees/1`; `absent` и `PENDING-VERIFY` ведут к отказу, а ослабление допустимо, только если оператор отдельным аутентифицированным действием принял его для этой задачи, станции, компенсации и срока.

Машину владелец квалифицирует сам воспроизводимой процедурой от обзора без привилегий до проверки с закрытым допуском. Аттестацию подписывает отдельный ключ машины, и она называет каждую из 42 гарантий ровно один раз. При дрейфе защищённых входов, смене класса платформы, проваленной проверке или отозванном доверии новые старты закрываются сразу, а работающее исполнение сужается или останавливается целиком не позже 300 с; повторы и перезапуски этот срок не сбрасывают.

4.5. Допуск к носителю

Учётку на носителе агент получает без общего секрета. Помощник с ключом машины слушает Unix-сокет, узнаёт UID вызывающего от ядра и подписывает только запрос на допуск с именем, разрешённым этому UID: метка `swarma-registrar registration/1`, 128-битное одноразовое число, срок не длиннее 120 с. Регистратор сверяет ключ машины и имя с реестром машин, проверяет свежесть числа и выдаёт пропуск, JWT с подписью Ed25519 на 120 с. В профиле Matrix сервер носителя принимает пропуск как вход `org.matrix.login.jwt`, доверяет только ключу регистратора, создаёт учётку при первом входе и выдаёт токен доступа, который сам аннулирует через 86 400 с, без refresh-токена; деактивированная учётка по пропуску не входит. Девять требований этого профиля к серверу квалифицируются на изолированном сервере до развёртывания сборки. Реестр машин состоит из записей `enrollment/1` в git, подписанных офлайновым издателем регистраций владельца. Отзыв машины требует двух действий: удалить её ключ из реестра и вывести все её учётки на сервере, а где этого требует квалифицированный путь сервера, деактивировать их.

4.6. Память как потребитель реестра

Память агентов хранит типизированные утверждения с источниками и обоснованиями. С реестром она стоит на одной подложке подписанных записей и читает его как потребитель: отправителя записи определяет по ячейке личности в текущем снимке, основание представительства берёт из гранта, а недоступный реестр никогда не считается выданным грантом. Полномочий память не выдаёт. Git-профиль памяти хранит пространства в git: агент сам подписывает запись, каждый читатель проверяет подпись, схему и право на ревизию при чтении, а содержимое ограничено нечувствительным, потому что из истории git ничего не стирается. Нормативный пакет 0.3.0 содержит 53 пункта ядра, 14 закрытых схем и 85 векторов соответствия: 77 смысловых и 8 для защищённого развёртывания.

4.7. Метод оркестрации: продьюсер и независимый ревьюер

Оркестратор декомпозирует цель, продьюсер производит изменение, ревьюер независимо проверяет опубликованную ревизию. На доске задач, трекере задач, запусков участников и вердиктов, запись Change Request (запрос на изменение) публикует неизменяемую ревизию, вердикт относится именно к ней, и один запуск не может принять собственную ревизию. Интеграция идёт через compare-and-set точного проверенного дерева. Изменения в CI, хуках, инструкциях агентов и ссылках одобряет детерминированный предикат над всей накопленной дельтой от базы до кандидата. Инструкции, хуки и MCP кандидата среда ревьюера не подхватывает.

В Swarma Protocol среда строит неизменяемый кандидат I над текущей головой M и фиксирует пару (M, I). Множество допустимых ревьюверов замораживается до появления хэша I, затем ревьюер вычисляется из хэшевой случайности кор-нод, и оркестратор не может выбрать удобного проверяющего после того, как кандидат известен. Одобрение и аттестации адресованы четвёрке из контракта, кандидата, политики и окружения. Сбой инструмента даёт аттестацию *не установлена* и зелёной проверкой не становится. Привилегированный класс изменений требует двух независимых ревью или одобрения человека из кор-команды. Независимость оценивается по административному домену, семейству модели и тулчейну. Оркестратор держит роль по аренде с эпохой и fencing-токеном [5, 25], и валидаторы отвергают запись устаревшей аренды.

5. Анализ безопасности

Гарантии ниже названы вместе с противником, аргументом, допущениями и пределом. Это аргументы уровня дизайна: каждая гарантия выведена из механизмов раздела 4 и держится, только если исполнитель эффекта вызывает проверку и физически удерживает область эффекта.

G1. Захваченный агент не получает закрытого ключа своей личности (П1). Материал ключа находится только у ключника, а личность выбирается по UID от ядра и активному поколению, поэтому ни имя в запросе, ни повторно выданный UID чужого ключа не дают. *Допущения:* ядро и root доверены, мастер-секрет цел. *Предел:* захваченный агент получает подписи собственной личности, и подпись удостоверяет авторство, правоты она не доказывает; расшифрованный пакет попадает в процесс адресата. Один ключ ключник не держит: в контракте брокера v0 ключ заявок вызывающего лежит в доме оркестратора, и его силу ограничивает грант, а запись git-профиля памяти ключник подписывает по

запросу агента. Компрометация мастер-секрета затрагивает всех агентов установки; восстановление идёт через новый корень и ячейку владельца, чьи ключи из мастер-секрета не выводятся.

G2. Делегирование не расширяет полномочий (П1, П2). Исполнитель проходит цепочку до корня и проверяет подписи, вложенность действий, целей и сроков и глубину. В общем языке грантов агент не выдаёт грантов даже в пределах своих прав, поэтому захваченный агент не раздаёт полученное соседям; вызывающий остаётся в цепочке потомка, и отзыв гранта оркестратора отрезает его агентов при следующей аренде или продлении. *Предел:* в контракте брокера v0 оркестратор, создающий другого оркестратора, подписывает ему суженный грант ключом вызывающего из своего дома. Захваченный оркестратор может так передать подмножество своих прав новым оркестраторам в пределах глубины и четырёх конвертов, и отзыв его гранта отрезает и их. Верный грант не делает владельца ключа добросовестным, и вредное действие в пределах гранта остаётся остаточным риском.

G3. Отзыв распространяется в ограниченный срок, неизвестность закрывает допуск (П1; П5 без ключа реестра). Снимок принимается только при возрасте не больше A, записи отзыва накапливаются, поэтому запись действует на каждой проверке не позже $\max(S + A, F)$; ответ *недоступно* запрещает эффект, а водяной знак не даёт шагу часов назад вернуть истёкшее право. *Допущения:* процесс подписи реестра и узел доверия не скомпрометированы, свидетель честен и опрашивается при каждой записи свежести. *Предел:* срок отсчитывается от подписанной публикации. Укравший ключ реестра может продлевать неизменный снимок записями свежести и не публиковать снимок с отзывом; потребитель, который отзыва не видел, тогда принимает отозванное полномочие без ограничения срока, и против этого действует только обнаружение по G4. Токен провайдера, уже выданный процессу, остаётся пригодным до аннулирования у провайдера. Завершение отзыва требует подтверждений названных потребителей или истечения прежних полномочий. Один полный снимок вмещает не больше 64 записей и 8192 байт.

G4. Откат и раздвоение журнала реестра обнаруживаются (П5). Контрольные точки пронумерованы и подписаны, свидетель фиксирует их и по запросу с вызовом называет последнюю голову, его доверенная точка хранится вне узла. *Допущения:* свидетель отделён от писателя реестра, хранитель внешней точки администрируется отдельно. Укравший ключ реестра не выпустит ни гранта, ни аттестации: их подписывают другие корни. *Предел:* он может задержать или скрыть публикацию и подписать ложный отзыв или ложную контрольную точку своей области. Реестр и свидетель на одной машине образуют один домен отказа. Владелец администрирует узел и может показать разным проверяющим разные истории одной личности; этот предел снимает только общее состояние Swarma Chain.

G5. Доставка, членство в комнате и подпись коллеги полномочий не дают (П2, П3). Ворота проверяют подпись над точными байтами, гранты, отзыв, окно времени и журнал повторов до доставки, а тело остаётся данными. Сохранение события до сдвига курсора и идемпотентный допуск исключают и потерю, и повторный допуск при падении моста. *Предел:* при допуске на уровне A0 компрометация сервера носителя или учётки человека даёт команды в пределах зоны этого человека, а агенты под одним пользователем ОС могут переписать свой корневой контекст. Decision guard инъекцию надёжно не распознаёт [12], подпись не доказывает истинности текста.

G6. Учётки на носителе ограничены меткой машины (П1, П4). Общего секрета нет, помощник подписывает только разрешённые этому UID имена, регистратор проверяет ключ машины по подписанному реестру, пропуск живёт 120 с, токен 86 400 с без продления. *Предел:* захваченный агент на зарегистрированной машине получает пропуск на своё разрешённое имя. Допуск закреплённого профиля сервера по времени продлевает пропуск до *exp* $+ 60$ с. Агентов под одним пользователем ОС помощник различает только по именам. Выданные токены живут до выхода, поэтому отзыв машины требует обоих действий.

G7. Исполнение допускается только на станции с квалифицированными гарантиями (П1). Диспетчер допускает пересечение запроса, минимумов платформы, политики владельца и текущих квалифицированных возможностей, а недостающую гарантию встречает типизированным отказом. Ослабление видно в матрице, в квитанции границы и в ответе планировщика и требует решения оператора. *Предел:* самоотчёт машины не защищает от захваченного root, подделанных наблюдений ядра и скопированного ключа машины. Профиль macOS не задаёт квалифицированного совокупного предела памяти. TLS внутри `CONNECT` не расшифровывается, поэтому разрешённый адресат остаётся каналом выноса, а периодические проверки сетевых правил оставляют окно обнаружения.

G8. Принятие относится к точному кандидату (П2). Одобрение и аттестации адресованы контракту, кандидату, политике и окружению, ревьювер вычисляется после фиксации кандидата, голова продвигается сравнением-и-заменой. Отключение любого из четырёх правил доставки в ограниченной модели даёт контрпример (раздел 6). *Предел:* правильность суждения не гарантируется. Модели склонны предпочитать собственный текст [35, 51], поэтому разные владельцы и семейства моделей уменьшают основания для корреляции ошибок, но независимости не доказывают. Квитанция скомпрометирован-

ной среды доказывает авторство отчёта и не доказывает его правдивости.

G9. Память не создаёт полномочий (П1, П2). У памяти нет операций одобрить, выдать грант, запустить и разбудить, ответ приходит агенту как данные с происхождением каждого элемента, запись при недоступном реестре отклоняется. *Предел:* из истории git ничего не стирается, а поиск секретов при записи их отсутствия не доказывает.

Захваченная машина (П4). Root на машине с ключником получает мастер-секрет установки или возможность им пользоваться, а с ним ключи личности и сообщений всех агентов установки и их гранты до отзыва; он получает и всё, что лежит на машине в файлах, включая ключи вызывающих в домах оркестраторов и программный корень брокера v0. Поэтому для этой установки G1, G7 и G2 в области брокера не держатся. Ключ оператора на телефоне, ключ реестра на узле доверия и офлайновый издатель регистраций остаются вне машины: противник не подписывает действий оператора, не подделывает контрольных точек и не регистрирует новых машин, а учётки на носителе ограничены именами этой машины (G6). Восстановление требует отзыва машины и нового корня установки.

Криптографический противник (П6). Подписи рантайма и протокола стоят на кривых, это Ed25519, P-256 в защищённых чипах, SSH, TLS и подписи git, поэтому против П6 ни одна из гарантий G1–G9 не держится. План миграции переводит долгоживущие корни (ячейку владельца, центр хостов, корень брокера) на хэш-подписи и оставляет ключи на кривых короткоживущими листьями. Ключи личности Swarma Chain используют SLH-DSA [33], а консенсус цепочки опирается только на хэш-функции.

6. Проверка: соответствие, квалификация, перебор и замеры

Оценка проверяет спецификации и отдельные компоненты по частям: согласие независимых реализаций на конечных корпусах, привилегированные шаги хоста, ограниченную модель интеграции, стоимость эталонной проверки записей и процесс ревью. Таблица 1 сводит свидетельства.

Доступность артефактов. Библиотека проверки подписанных записей с корпусом опубликована в организации github.com/relux-works; корпус проверен на ревизии f76a400, замеры сделаны на её предке 9b42338. Пакет спецификации памяти лежит в каталоге `spec/` репозитория relux-works/swarma-memory (ревизия 763e024), запись о привилегированных запусках находится в `docs/root-qualification-evidence.md` репозитория relux-works/swarma-user-manager. Модель интеграции приложена к исходникам этой работы. Обвязка замеров, отчёт о замерах, записи ревью и книга [52] не опубликованы.

6.1. Корпуса соответствия

В корпусе подписанных записей каждый случай отказа несёт точный ожидаемый код и ссылку на положительный контроль, поэтому реализация, которая отказывает всегда, корпус не проходит. Генератор на Go пишет случаи и ожидания, проверка на Python сама канонизирует байты и проверяет подписи Ed25519 и P-256, вердиктов Go не импортируя. В репозитории лежат 944 именованных случая, а 1688 механических развёрток материализуются по манифесту с хэшами SHA-256. Тест библиотеки генерирует полный корпус, сверяет манифест и требует точного кода и плана для всех 2632 случаев. Полный прогон проверки на Python на всех случаях сообщает репозиторий. Мы повторили тест библиотеки на публичной ревизии f76a400 библиотеки (Go 1.26.0, darwin/arm64): 2632 подтеста прошли, код выхода 0. Процедура корпуса задаёт и отрицательный контроль самих инструментов: изменённый байт подписи в положительном случае обязан дать расхождение у генератора и код `envelope_signature_invalid` у проверки на Python.

Для памяти первый вычислитель написан по сводному контракту ядра без доступа к пакету и прогнан на векторах пакета без их правки, второй написан только по пакету, и каждое расхождение разобрано как ошибка пакета, реализации или неоднозначность контракта. На корпусе 0.2.7 оба дают ожидаемые ответы на всех 66 смысловых векторах; на пакете 0.3.0 вычислитель на Go проходит все 77 смысловых векторов.

6.2. Привилегированные шаги хоста станции

Привилегированные шаги проверены двумя запусками под root на одной машине Linux, каждый на своей сборке. Первый прошёл режимы применения, проверки и отката для одноразовых примитивов учёток: команды учёток и групп, защита дома, отказ ACL и capabilities, остановка через pidfd и ограниченный откат. Второй установил набор квалификации: при применении квалификация в режиме загрузки прошла 11 из 11 проверок, отдельная проверка тоже прошла 11 из 11; допуск станций в этом запуске не настраивался. Опубликованная запись о результатах ограничивает охват второго запуска: он не включает кампании перезагрузок и отката установки и не исполняет проверку открытых inode и пространств имён. Набор из 11 проверок отличается от самоквалификации раздела 4.4 и подписанной аттестации по 42 гарантиям не выпускает.

Таблица 1. Сводка свидетельств. Каждая строка относится к названной ревизии и названным условиям.

Утверждение	Метод	Результат	Чего не показывает
Генератор и две независимые проверки подписанных записей согласуются	Корпус соответствия: генератор на Go, независимая проверка на Python, библиотека на Go; у отказа точный код и положительный контроль	2632 случая (2499 отрицательных, 133 положительных); согласие трёх сообщает репозиторий, библиотеку мы повторили	Однозначности и полноты спецификации, корректности вне корпуса, работы проверки внутри служб
Два вычислителя ядра памяти согласуются	Два вычислителя: один по сводному контракту без доступа к пакету, второй по пакету	Ожидаемые ответы на всех 66 смысловых векторах у обоих	Однозначности контракта, защищённой службы (8 векторов развёртывания не исполняются), пользы памяти
Привилегированные шаги хоста исполнимы на Linux	Запуски под root на одной машине, привязанные к сборкам	Примитивы учёток: применение, проверка, откат; квалификационный набор: 11 из 11 проверок при применении и при проверке	Самоквалификации по 42 гарантиям, допуска станций, нагрузки, перезагрузок, отката установки, других машин
Четыре правила доставки необходимы для свойства интеграции	Исчерпывающий перебор ограниченной модели с отрицательным контролем на каждое правило	1090 состояний без нарушения; без любого правила найден контрпример	Неограниченных параметров, эквивалентности реализации, честности среды
Стоимость эталонной проверки одной записи	Микрозамеры эталонной библиотеки на зафиксированных входах в одном потоке Go	Медианы от 2,1 до 18,0 мс и от 1,9 до 18,9 млн байт выделений на вызов (табл. 2)	Задержки и пропускной способности системы, худшего случая, оптимизированной реализации
Ревьювер возвращает заметную долю первых ревизий	Разбор записей ревью на доске задач одного проекта	Приняты 17 из 35 разобранных первых раундов; медиана 2 раунда; вердикт разобран в 104 из 318 записей	Пропущенных дефектов, точности ревьюверов, причинного эффекта, несмещённых долей

Таблица 2. Эталонная проверка записей на вызов. Время дано медианой и размахом [мин.–макс.] пяти повторов; размах не является доверительным интервалом. Память (10^6 байт) и число выделений даны медианой.

Вызов и вход	Время, мс	Память	Выделений
Сообщение `message/1`, 835 байт	6,04 [5,70–7,22]	6,30	92 619
Грант, одно звено, 832 байта	9,10 [7,76–10,53]	9,00	132 307
Грант, три звена, 2628 байт	17,99 [15,53–23,75]	18,91	278 351
Аттестация ключа сообщений, 840 байт	3,78 [3,71–4,51]	4,49	66 016
Снимок отзывов на 64 записи, 7110 байт	2,11 [1,67–2,72]	1,94	28 497
Журнал из 64 записей и две контрольные точки, 30 718 байт	9,21 [7,54–11,23]	7,53	112 915

6.3. Ограниченная модель интеграции

Модель в `paper/model/` воспроизводится командой `make -C paper model`. Она содержит две задачи, до двух версий контракта, до двух эпох аренды и до двух доставок записи и на границе доставки применяет четыре правила: голова равна базе записи (`cas`), запись несёт текущую эпоху (`epoch`), аттестация ссылается на текущий контракт (`contract`) и адресована точному кандидату (`bind_I`). При всех правилах перебор в ширину достигает 1090 состояний без нарушения. Без `cas` запись применяется дважды после 84 просмотренных состояний, без `epoch` устаревшая эпоха продвигает голову после 82, без `contract` аттестация ссылается на заменённый контракт после 71, без `bind_I` аттестация сделана на иной базе после 490. Эти числа отмечают остановку поиска и безопасность вариантов не сравнивают.

6.4. Микрозамеры эталонной проверки записей

Мы измерили шесть публичных вызовов библиотеки проверки на ревизии 9b42338, на неизменённых входах корпуса; реализация эталонная и не оптимизировалась. Замер шёл в гостевой системе KVM на процессоре AMD Ryzen 7 9700X (гостю видны 8 логических CPU, без закрепления), ядро Linux 7.0.0-38-generic, Go 1.26.10 linux/amd64, `GOMAXPROCS=1`, пять последовательных повторов стандартного бенчмарка Go с `-benchmem`. Машина была общей: одноминутная средняя нагрузка составляла 1,73 до замера и 1,95 после. Внутри таймера библиотека разбирает и проверяет сырые подписанные байты, и каждая итерация проверяет, что вызов принят; загрузка фикстур, ввод-вывод и подпись в замер не входят.

Разница одного и трёх звеньев не оценивает влияния глубины: входы различаются содержимым, и в

трёхзвенной цепочке есть подпись P-256. Все записи снимка отзывов вступают в силу позже момента проверки, поэтому худший поиск среди действующих отзывов не измерен. Вызов журнала сравнивает вторую контрольную точку на раздвоение, но её суффикса не проверяет. `GOMAXPROCS=1` даёт Go один поток исполнения и ядра не закрепляет. Реализация много выделяет: от 28 до 278 тысяч выделений и от 1,9 до 18,9 млн байт на вызов. При медиане 6,04 мс на сообщение один поток проверяет около 165 сообщений в секунду, и для ворот, которые проверяют каждое событие, это стоимость эталонной реализации; оптимизированная реализация не измерялась. Накладных расходов служб, ввода-вывода и сети замер не включает.

6.5. Статистика ревью

В методе раздела 4.7 ревьювер выносит вердикт с находками по классам серьёзности. Мы разобрали файлы исходов ревью на доске задач проекта Swarma Curator, куратора из раздела 1, по состоянию на 24 октября 2026 года. Доска рабочая, и её записи не опубликованы. Вердиктом считалось последнее поле `verdict` в файле, серьёзности брались из JSON находок, записи без вердикта учитывались только в раундах. Нашлось 140 задач с записями ревью и 318 записей. Вердикт разобран в 104 записях из 318, около трети; остальные 214 написаны в ранних форматах, поэтому выборка смещена к поздним записям. Из 104 разобранных вердиктов 55 *принято* и 49 *нужны изменения*; из 35 разобранных первых раундов принято 17. Медиана числа раундов на задачу равна 2, максимум 13. Находок высшего класса серьёзности P0 нет, классов P1 и P2 соответственно 73 и 49. Второй разборщик текстовых вердиктов даёт 58 и 51 при 209 неразобранных записях. Другие проекты не учтены. Числа задач, записей и находок служат нижней границей для этой доски; медиана, доля принятых первых раундов и соотношение вердиктов нижней границей не являются, и пропущенные записи могут сдвинуть их в любую сторону. Числа показывают, что ревьювер регулярно возвращает работу, и ничего не говорят о дефектах, которые он пропустил.

6.6. Ограничения

Гарантии раздела 5 держатся при допущениях раздела 3. Три предела внутри одной установки не компенсируются: владелец узла доверия может показать разным проверяющим разные истории одной личности, и это снимает только Swarma Chain (G4); укравший ключ реестра может бессрочно скрывать отзыв от потребителя, который его не видел (G3); root машины с ключником получает ключи всех агентов установки (П4). Правильности суждения модели, в том числе ревьювера и decision guard, платформа не обещает, а отказ в обслуживании не предотвращает.

Оценка измеряет компоненты и спецификации, а свойства системы целиком выводит из дизайна. Граница 300 с и формула $\max(S + A, F)$ выведены аналитически: задержку от отзыва гранта или квалификации через реестр, брокер, диспетчер и хост сессий до завершённого сужения или остановки оценка не измеряет. Замеры относятся к библиотеке на горячих входах и о производительности системы целиком не говорят. Инъекции, захват машины и атаки на узел доверия против работающей установки оценка не испытывает, пользу роя и памяти на внешних задачах не измеряет. Привилегированные шаги выполнены на одной машине, а модель интеграции не описывает аренды, остановки и внешних эффектов.

7. Смежные работы

Полномочия. Гранты продолжают линию capability-систем [13, 32]: право передаётся вместе со ссылкой и только сужается. Macaroons [4] добавляют ограничения к токену без обращения к издателю, Biscuit [15] даёт публично проверяемые токены с офлайн-ослаблением и языком правил наподобие Datalog, SPKI/SDSI [16] и UCAN [18] строят цепочки делегирования на открытых ключах. Swarma заимствует ослабление и проверку у получателя и добавляет зависимость потомка от вызывающего и отзыв подписанными снимками с явной границей срока.

Прозрачность ключей. Certificate Transparency [27, 28] и CONIKS [31] делают раздвоение журнала сертификатов и каталога ключей обнаружимым деревьями Меркла и аудиторами, Sigstore [34] применяет журнал прозрачности к подписям артефактов. Реестр владельца решает более узкую задачу одного владельца, и раздвоение перед разными проверяющими снимает только общее состояние цепочки.

Самосертифицирующие идентификаторы. KERI [39] вводит журнал ключевых событий и предротацию со следующими ключами за хэшем. Swarma использует ту же идею для ячейки личности и выводит ключи агентов иерархически [46, 20], храня ключи владельца отдельно от мастер-секрета.

Мессенджеры. Перекрёстная подпись Matrix [40] и MLS [2] удостоверяют устройства и клиентов и дают групповое сквозное шифрование. В профиле Matrix Swarma использует перекрёстную подпись как уровень A1 и добавляет поверх носителя подпись точных байтов, гранты и ворота доверия: подлинное устройство права на действие не означает.

Изоляция. gVisor (его стоимость измерена в сторонней работе [50]), Firecracker [1] и Kata Containers [24] изолируют недоверенный код ядром в пространстве пользователя или лёгкими виртуальными машинами. Станция опирается на пользователя ОС, группу ресурсов и файрвол ядра без образа окружения, а более сильные классы, например класс на виртуальной машине без выбора основы, входят через тот же контракт изоляции после отдельной квалификации; публичный пул станцией не служит.

Агентские системы. ReAct [48] связал рассуждение с вызовом инструментов, SWE-agent [47] и SWE-bench [22] перенесли это на разработку, AutoGen, MetaGPT и ChatDev [44, 21, 36] организуют несколько ролей и обмен между ними. Параллельные агенты, написавшие компилятор [6], опирались на тесты как оракул, а в Prove2Me [8] работу агентов независимых владельцев принимает ядро доказательств. Эти работы изучают способности и координацию, а ключи, полномочия и отзыв оставляют среде.

Системная защита агентов от инъекций. Инъекция против агентов с инструментами описана в [19] и измерена в AgentDojo [12]. Ближайшие к Swarma работы выносят полномочия из модели. Шаблон двух моделей [43] не показывает недоверенный текст модели с правом вызывать инструменты. CaMeL [11] извлекает поток управления из доверенного запроса и проверяет политику на capabilities данных при вызове инструмента, FIDES [10] отслеживает метки конфиденциальности и целостности в планировщике и детерминированно применяет политику, Progent [38] задаёт агенту политику привилегий на вызовы инструментов, IsolateGPT [45] исполняет приложения на базе модели изолированно и опосредует их взаимодействие, а [3] сводит такие решения в шаблоны проектирования. С ними Swarma разделяет два принципа: допуск решает детерминированный код, а оценка моделью только сужает. Эти работы ограничивают потоки данных и вызовы инструментов внутри одного агентского приложения. Swarma ставит проверку у исполнителя эффекта за границей процесса: личность агента задаёт ядро через учётку ОС, полномочия передаются подписанными грантами между машинами и владельцами с ограниченным сроком отзыва, а исполнение удерживает станция. Механизмы этих работ можно применить внутри станции как дополнительное сужение.

Ревью кода моделями. Автоматизация ревью изучалась на глубоких моделях [42], на предобученных [41, 29] и в промышленном исправлении замечаний [17]. Модели в роли судьи систематически смещены, в том числе к собственному тексту [51, 35]; отсюда оценка независимости ревьювера по домену, семейству модели и тулчейну. Журнал протокола следует BFT-протоколам [7, 49], а пределы репутации и множественных личностей описаны в [14, 9, 23].

8. Заключение и открытые вопросы

Swarma переносит решение о допуске из суждения модели в детерминированные проверки у исполнителя эффекта. Захваченный агент не владеет ключом, не расширяет грант делегированием, не получает прав от доставки сообщения и не исполняется на станции с неподтверждёнными гарантиями. Доверие сводится к ядру, root и доверенной базе хоста, к целому мастер-секрету ключника, к проверенному источнику времени, к узлу доверия под администрированием владельца, к человеку с аппаратным ключом и, в протоколе, к целостности среды проекта; у каждой гарантии названы допущения и предел.

Аргумент дизайна оставляет эмпирические вопросы. Первый касается задержки отзыва гранта и квалификации в сквозном прогоне станции с реестром и свидетелем на узле доверия и поведения границ против программных нарушителей. Следующие касаются стойкости работающей установки к инъекциям, пользы роя по сравнению с одиночным агентом на одинаковых внешних задачах и полноты ревью, которую калибрует посев дефектов. Для протокола тот же вопрос ставит механически проверяемая спецификация аренды, остановки и внешних эффектов [26]. Состояние реализации каждой части описано в отдельном приложении *Swarma: состояние реализации*.

Список литературы

[1] A. Agache, M. Brooker, A. Florescu, et al. Firecracker: Lightweight virtualization for serverless applications. In *17th USENIX Symposium on Networked Systems Design and Implementation (NSDI 20)*, pages 419–434. USENIX Association, 2020.

[2] R. Barnes, B. Beurdouche, R. Robert, J. Millican, E. Omara, and K. Cohn-Gordon. The messaging layer security (MLS) protocol. RFC 9420, IETF, July 2023.

[3] L. Beurer-Kellner, B. Buesser, A.-M. Creţu, E. Debenedetti, D. Dobos, D. Fabian, M. Fischer, D. Froelicher, K. Grosse, D. Naeff, E. Ozoani, A. Paverd, F. Tramèr, and V. Volhejn. Design patterns for securing LLM agents against prompt injections. arXiv:2506.08837, 2025.

[4] A. Birgisson, J. G. Politz, Ú. Erlingsson, A. Taly, M. Vrable, and M. Lentczner. Macaroons: Cookies with contextual caveats for decentralized authorization in the cloud. In *2014 Network and Distributed System Security Symposium (NDSS)*. Internet Society, 2014.

[5] M. Burrows. The Chubby lock service for loosely-coupled distributed systems. In *7th USENIX Symposium on Operating Systems Design and Implementation (OSDI 06)*, pages 335–350. USENIX Association, 2006.

[6] N. Carlini. Building a C compiler with a team of parallel Claudes. `https://www.anthropic.com/engineering/building-c-compiler`, 2026. Anthropic Engineering blog, 5 February 2026.

[7] M. Castro and B. Liskov. Practical Byzantine fault tolerance. In *3rd Symposium on Operating Systems Design and Implementation (OSDI 99)*. USENIX Association, 1999.

[8] S. Chen, K. Marwaha, X. Lu, H. Yuen, and T. Peng. Prove2Me: An open collaborative platform for scaling math formalization. arXiv:2608.28433, 2026.

[9] A. Cheng and E. Friedman. Sybilproof reputation mechanisms. In *2005 ACM SIGCOMM Workshop on Economics of Peer-to-Peer Systems (P2PECON '05)*. ACM, 2005.

[10] M. Costa, B. Köpf, A. Kolluri, A. Paverd, M. Russinovich, A. Salem, S. Tople, L. Wutschitz, and S. Zanella-Béguelin. Securing AI agents with information-flow control. arXiv:2505.23643, 2025.

[11] E. Debenedetti, I. Shumailov, T. Fan, J. Hayes, N. Carlini, D. Fabian, C. Kern, C. Shi, A. Terzis, and F. Tramèr. Defeating prompt injections by design. arXiv:2503.18813, 2025.

[12] E. Debenedetti, J. Zhang, M. Balunović, L. Beurer-Kellner, M. Fischer, and F. Tramèr. AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2024.

[13] J. B. Dennis and E. C. Van Horn. Programming semantics for multiprogrammed computations. *Communications of the ACM*, 9(3):143–155, Mar. 1966.

[14] J. R. Douceur. The Sybil attack. In *Peer-to-Peer Systems: First International Workshop (IPTPS 2002)*, volume 2429 of *Lecture Notes in Computer Science*, pages 251–260. Springer, 2002.

[15] Eclipse Biscuit project. Biscuit, a bearer token with offline attenuation and decentralized verification. `https://github.com/eclipse-biscuit/biscuit/blob/v3.3/SPECIFICATIONS.md`, 2024. Specification, version 3.3. Accessed 2026-10-25.

[16] C. M. Ellison, B. Frantz, B. Lampson, R. Rivest, B. Thomas, and T. Ylonen. SPKI certificate theory. RFC 2693, IETF, Sept. 1999.

[17] A. Froemmgen, J. Austin, P. Choy, et al. Resolving code review comments with machine learning. In *46th International Conference on Software Engineering, Software Engineering in Practice (ICSE-SEIP)*, pages 204–215. ACM, 2024.

[18] I. Gozalishvili, D. Holmgren, P. Krüger, and B. Zelenka. User controlled authorization network (UCAN) specification, version 1.0.0. `https://github.com/ucan-wg/spec`, 2026. README at commit 8072f3f; no release tag.

[19] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz. Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. In *16th ACM Workshop on Artificial Intelligence and Security (AISec)*, pages 79–90. ACM, 2023.

[20] J. Hoenicke and P. Rusnak. SLIP-0010: Universal private key derivation from master private key. `https://github.com/satoshilabs/slips/blob/master/slip-0010.md`, 2016. SatoshiLabs Improvement Proposal. Accessed 2026-10-25.

[21] S. Hong, M. Zhuge, J. Chen, et al. MetaGPT: Meta programming for a multi-agent collaborative framework. In *International Conference on Learning Representations (ICLR)*, 2024.

[22] C. E. Jimenez, J. Yang, A. Wettig, et al. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations (ICLR)*, 2024.

[23] S. D. Kamvar, M. T. Schlosser, and H. Garcia-Molina. The EigenTrust algorithm for reputation management in P2P networks. In *Twelfth International Conference on World Wide Web (WWW '03)*. ACM, 2003.

[24] Kata Containers project. Kata Containers: An OpenInfra Foundation project. `https://katacontainers.io/`, 2026. Accessed 2026-10-25.

[25] M. Kleppmann. How to do distributed locking. `https://martin.kleppmann.com/2016/02/08/how-to-do-distributed-locking.html`, 2016. Blog post, 8 February 2016.

[26] L. Lamport. *Specifying Systems: The TLA+ Language and Tools for Hardware and Software Engineers*. Addison-Wesley, 2002.

[27] B. Laurie, A. Langley, and E. Kasper. Certificate transparency. RFC 6962, IETF, June 2013.

[28] B. Laurie, E. Messeri, and R. Stradling. Certificate transparency version 2.0. RFC 9162, IETF, Dec. 2021.

[29] Z. Li, S. Lu, D. Guo, et al. Automating code review activities by large-scale pre-training. In *30th ACM Joint ESEC/FSE*, pages 1035–1047. ACM, 2022.

[30] D. Malkhi and M. Reiter. Byzantine quorum systems. *Distributed Computing*, 11(4):203–213, 1998.

[31] M. S. Melara, A. Blankstein, J. Bonneau, E. W. Felten, and M. J. Freedman. CONIKS: Bringing key transparency to end users. In *24th USENIX Security Symposium (USENIX Security 15)*, pages 383–398. USENIX Association, 2015.

[32] M. S. Miller. *Robust Composition: Towards a Unified Approach to Access Control and Concurrency Control*. PhD thesis, Johns Hopkins University, May 2006.

[33] National Institute of Standards and Technology. Stateless hash-based digital signature standard. FIPS 205, NIST, Aug. 2024.

[34] Z. Newman, J. S. Meyers, and S. Torres-Arias. Sigstore: Software signing for everybody. In *2022 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, pages 2353–2367. ACM, 2022.

[35] A. Panickssery, S. R. Bowman, and S. Feng. LLM evaluators recognize and favor their own generations. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

[36] C. Qian, W. Liu, H. Liu, et al. ChatDev: Communicative agents for software development. In *62nd Annual Meeting of the ACL (Volume 1: Long Papers)*, pages 15174–15186. Association for Computational Linguistics, 2024.

[37] P. Saint-Andre. Extensible messaging and presence protocol (XMPP): Core. RFC 6120, IETF, Mar. 2011.

[38] T. Shi, J. He, Z. Wang, H. Li, L. Wu, W. Guo, and D. Song. Progent: Securing AI agents with privilege control. arXiv:2504.11703, 2025.

[39] S. M. Smith. Key event receipt infrastructure (KERI). arXiv:1907.02143, 2019.

[40] The Matrix.org Foundation. Matrix specification v1.19: Client-Server API, section end-to-end encryption, cross-signing. `https://spec.matrix.org/v1.19/client-server-api/`, 2026. Released 2026-07-08. Accessed 2026-10-25.

[41] R. Tufano, S. Masiero, A. Mastropaolo, L. Pascarella, D. Poshyvanyk, and G. Bavota. Using pre-trained models to boost code review automation. In *44th International Conference on Software Engineering (ICSE)*, pages 2291–2302. ACM, 2022.

[42] R. Tufano, L. Pascarella, M. Tufano, D. Poshyvanyk, and G. Bavota. Towards automating code review activities. In *43rd IEEE/ACM International Conference on Software Engineering (ICSE)*, pages 163–174. IEEE, 2021.

[43] S. Willison. The dual LLM pattern for building AI assistants that can resist prompt injection. `https://simonwillison.net/2023/Apr/25/dual-llm-pattern/`, 2023. Blog post, 25 April 2023.

[44] Q. Wu, G. Bansal, J. Zhang, et al. AutoGen: Enabling next-gen LLM applications via multi-agent conversation. arXiv:2308.08155, 2023.

[45] Y. Wu, F. Roesner, T. Kohno, N. Zhang, and U. Iqbal. IsolateGPT: An execution isolation architecture for LLM-based agentic systems. In *Network and Distributed System Security Symposium (NDSS)*. Internet Society, 2025.

[46] P. Wuille. BIP 32: Hierarchical deterministic wallets. `https://github.com/bitcoin/bips/blob/master/bip-0032.mediawiki`, 2012. Bitcoin Improvement Proposal. Accessed 2026-10-25.

[47] J. Yang, C. E. Jimenez, A. Wettig, et al. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

[48] S. Yao, J. Zhao, D. Yu, et al. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023.

[49] M. Yin, D. Malkhi, M. K. Reiter, G. G. Gueta, and I. Abraham. HotStuff: BFT consensus with linearity and responsiveness. In *2019*

ACM Symposium on Principles of Distributed Computing (PODC), pages 347–356. ACM, 2019.

[50] E. G. Young, P. Zhu, T. Caraza-Harter, A. C. Arpaci-Dusseau, and R. H. Arpaci-Dusseau. The true cost of containing: A gVisor case study. In *11th USENIX Workshop on Hot Topics in Cloud Computing (HotCloud 19)*. USENIX Association, 2019.

[51] L. Zheng, W.-L. Chiang, Y. Sheng, et al. Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2023.

[52] Алексей Григорьев и Иван Опарин. Swarma. Платформа для роёв ИИ-агентов: от одной машины до открытой сети. Книга, версия 1.7.1, рукопись, 2026. Не опубликована.

[53] Алексей Григорьев и Иван Опарин. Swarma: платформа для роёв автономных агентов. Сопутствующая работа, версия 7.1.0, рукопись, 2026. Не опубликована.